

SET A — Answer Key & Concepts

(CONFIDENTIAL)

Each entry lists the bugs and the fixed program. Use the fixed program to generate expected outputs for the hidden test cases.

M1 (Python)

Concept explained: A single artificial neuron computes $z = \sum w_i x_i + b$ and then applies the sigmoid $\sigma(z) = 1/(1+e^{-z})$, which squashes z into $(0,1)$. It is the building block of neural networks and logistic regression. In Python, `z + b` on its own line computes a value and discards it. Use `round()` only for numbers, and an f-string for fixed decimals.

1. `range(1, n)` → `range(n)` (skips the first weight)
2. `z + b` does nothing → `z += b`
3. Sigmoid is `1/(1+e^-z)`, and `round` can drop trailing zeros, so use format `:.4f`

```
import math
n = int(input())
w = list(map(int, input().split()))
x = list(map(int, input().split()))
b = int(input())
z = 0
for i in range(n):
    z += w[i] * x[i]
z += b
print(f"1 / (1 + math.exp(-z)):.4f")
```

M2 (C++)

Concept explained: Binary search halves a sorted range $[lo, hi]$ on each step, which takes $O(\log n)$ time. To find the FIRST occurrence, record the match and keep searching LEFT ($hi = mid-1$). With inclusive bounds, the loop must run while $lo \leq hi$ and hi must start at $n-1$.

1. `hi = n` → `hi = n - 1`
2. `lo < hi` → `lo <= hi`
3. On a match, keep searching **left**: `hi = mid - 1`

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, t; cin >> n >> t;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int lo = 0, hi = n - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) { ans = mid; hi = mid - 1; }
        else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    cout << ans << endl;
}

```

M3 (Java)

Concept explained: The Caesar cipher shifts every letter k places: $E(x) = (x + k) \bmod 26$, where $x = c - 'a'$. The result is a position from 0 to 25, so it must be converted back to a character by adding 'a' or 'A'. In Java, % of a negative number is negative, so normalize k with $((k \% 26) + 26) \% 26$.

1. Negative k : $k = ((k \% 26) + 26) \% 26$
2. Lowercase result is missing the 'a' + offset
3. Uppercase branch uses $c - 'a' \rightarrow c - 'A'$

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int k = Integer.parseInt(sc.nextLine().trim());
        String s = sc.nextLine();
        k = ((k % 26) + 26) % 26;
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isLowerCase(c))
                sb.append((char) ('a' + (c - 'a' + k) % 26));
            else if (Character.isUpperCase(c))
                sb.append((char) ('A' + (c - 'A' + k) % 26));
            else
                sb.append(c);
        }
        System.out.println(sb);
    }
}

```

M4 (Python)

Concept explained: A moving average smooths a noisy signal and is the simplest FIR low-pass filter. Instead of recomputing each window's sum, add the incoming sample $a[i]$ and subtract the outgoing one $a[i-k]$, which is $O(n)$ overall. `//` is floor division, so use `/` for averages.

1. `range(k, n - 1) → range(k, n)` (misses the last window)

2. The outgoing element is `a[i - k]`
3. `s // k → s / k`

```
n, k = map(int, input().split())
a = list(map(int, input().split()))
s = sum(a[:k])
res = [s / k]
for i in range(k, n):
    s += a[i] - a[i - k]
    res.append(s / k)
print(" ".join(f"{v:.2f}" for v in res))
```

M5 (C++)

Concept explained: XOR gives 1 exactly where two bits differ, so the Hamming distance is `popcount(x ^ y)`. It is used in error-detecting and error-correcting codes. Check the LSB with `& 1` and move to the next bit with `>>= 1`. The expression `d << 1` on its own changes nothing.

1. `x & y → x ^ y`
2. `d << 1` does nothing and shifts the wrong way → `d >>= 1` (infinite loop otherwise)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long x, y; cin >> x >> y;
    long long d = x ^ y;
    int c = 0;
    while (d > 0) {
        c += d & 1;
        d >>= 1;
    }
    cout << c << endl;
}
```

M6 (Java)

Concept explained: Two pointers move inward from both ends. Characters that aren't letters or digits must be skipped with `continue` before comparing, and the comparison must ignore case. Java's `Scanner.next()` reads one token only, while `nextLine()` reads the whole line.

1. `sc.next()` reads only one word → `sc.nextLine()`
2. After skipping a non-alphanumeric character, the loop must `continue` instead of comparing
3. The comparison must ignore case

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (!Character.isLetterOrDigit(s.charAt(i))) { i++; continue; }
            if (!Character.isLetterOrDigit(s.charAt(j))) { j--; continue; }
            if (Character.toLowerCase(s.charAt(i)) != Character.toLowerCase(s.charAt(j)))
        { System.out.println("NO"); return; }
            i++; j--;
        }
        System.out.println("YES");
    }
}
```

M7 (Python)

Concept explained: $C = A \times B$ with $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$. A is $n \times m$ and B is $m \times p$, so C is $n \times p$. This operation is at the core of neural network layers. In Python, `[[0]*p]*n` repeats the SAME row object n times, so changing one row changes all of them. Use a list comprehension instead.

1. `[[0]*n]*n` creates aliased rows
2. Wrong shape: C is $n \times p$
3. `B[j][k] → B[k][j]`

```
n, m, p = map(int, input().split())
A = [list(map(int, input().split())) for _ in range(n)]
B = [list(map(int, input().split())) for _ in range(m)]
C = [[0] * p for _ in range(n)]
for i in range(n):
    for j in range(p):
        for k in range(m):
            C[i][j] += A[i][k] * B[k][j]
for row in C:
    print(*row)
```

M8 (C++)

Concept explained: Repeated division by 2 produces the remainders, which are the bits from LSB to MSB, so the string must be reversed at the end. `char(1)` is a control character, while '0' + bit gives the digit character. 0 needs special handling.

1. `n > 1 → n > 0` (drops the MSB)
2. `char(n % 2) → char('0' + n % 2)`
3. Bits are produced LSB-first, so reverse the string
4. `n == 0` must print 0

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    if (n == 0) { cout << 0 << endl; return 0; }
    string s;
    while (n > 0) {
        s += char('0' + n % 2);
        n /= 2;
    }
    reverse(s.begin(), s.end());
    cout << s << endl;
}
```

M9 (Java)

Concept explained: Push opening brackets. For a closing bracket, the stack top must be the matching opener. Popping an empty stack throws an exception in Java. The string is balanced only if the stack is empty at the end. Compilers use the same idea to parse expressions.

1. `pop()` on an empty stack throws an exception, so return false if it is empty
2. `]` must match `[`, not `(`
3. At the end, the stack must be empty

```
import java.util.*;
public class Main {
    static boolean ok(String s) {
        Deque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') st.push(c);
            else {
                if (st.isEmpty()) return false;
                char top = st.pop();
                if ((c == ')') && top != '(') || (c == ']') && top != '[') || (c == '}') &&
top != '{'))
                    return false;
            }
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = Integer.parseInt(sc.nextLine().trim());
        while (t-- > 0) System.out.println(ok(sc.nextLine().trim()) ? "YES" : "NO");
    }
}
```

M10 (Python)

Concept explained: 1-NN predicts the label of the closest training point. Euclidean distance is $\sqrt{\sum (p_i - q_i)^2}$. Using `abs` instead of squares gives a different metric. The minimum must start at $+\infty$ and be updated with `<`.

1. Euclidean distance needs `(p - q) ** 2`, not `abs`
2. `best` must start at infinity

3. `dist > best` → `dist < best` (strict, so the earliest point wins a tie)

```
n, d = map(int, input().split())
train = []
for _ in range(n):
    parts = input().split()
    train.append((list(map(int, parts[:d])), parts[d]))
query = list(map(int, input().split()))
best, label = float("inf"), None
for pt, l in train:
    dist = sum((p - q) ** 2 for p, q in zip(pt, query)) ** 0.5
    if dist < best:
        best, label = dist, l
print(label)
```

M11 (C++)

Concept explained: Mark multiples of every prime p starting from p^2 , up to and including N . This takes $O(N \log \log N)$ time. The array needs $N+1$ entries so that index N exists, and 0 and 1 are not prime. Large primes are what make RSA keys secure.

1. The vector size must be `n + 1` (index n is out of bounds)
2. `i * i < n` → `<= n`
3. `j < n` → `j <= n`
4. Counting must start at 2 (0 and 1 aren't prime)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<bool> isP(n + 1, true);
    for (int i = 2; (long long)i * i <= n; i++)
        if (isP[i])
            for (int j = i * i; j <= n; j += i) isP[j] = false;
    int c = 0;
    for (int i = 2; i <= n; i++) if (isP[i]) c++;
    cout << c << endl;
}
```

M12 (Java)

Concept explained: $F(n) = F(n-1) + F(n-2)$ is computed iteratively. Apply mod $1e9+7$ to the sum: `(a+b) % MOD`, because `%` binds tighter than `+`. Two values near $1e9$ add up to more than the int limit ($2.1e9$), so use long. After n iterations, `a` holds $F(n)$.

1. `int` overflows when adding two values near MOD, so use `long`
2. Operator precedence: `(a + b) % MOD`
3. After n steps, `a` holds $F(n)$: print `a`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        long MOD = 1_000_000_007L;
        int n = new Scanner(System.in).nextInt();
        long a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            long c = (a + b) % MOD;
            a = b; b = c;
        }
        System.out.println(a);
    }
}
```

M13 (Python)

Concept explained: An even-parity bit makes the total count of 1s even, so the receiver can detect any single-bit error. If the number of 1s is odd, the parity bit is 1. In Python, `s.count(1)` passes an int where a str is needed, and `str + int` raises a `TypeError`.

1. `s.count(1) → s.count('1')` (`TypeError`)
2. Parity logic is reversed: an odd count needs a 1
3. Concatenation needs `str(p)`

```
t = int(input())
for _ in range(t):
    s = input().strip()
    ones = s.count('1')
    p = 1 if ones % 2 == 1 else 0
    print(s + str(p))
```

M14 (C++)

Concept explained: Kadane's algorithm: `cur = max(x, cur + x)` is the best subarray ending at `x`, and `best` is the maximum of all `cur` values. The answer must allow an all-negative array, so `best` starts at `a[0]`, not 0. Sums can reach 10^{14} , so use `long long`.

1. `best = 0` is wrong when all values are negative, so start from `a[0]`
2. `best = max(best, x) → max(best, cur)`
3. Sums reach 10^{14} , so use `long long`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> a(n);
    for (auto &v : a) cin >> v;
    long long best = a[0], cur = 0;
    for (long long x : a) {
        cur = max(x, cur + x);
        best = max(best, cur);
    }
    cout << best << endl;
}
```

M15 (Java)

Concept explained: A strong password needs ALL of its rules to hold, so the conditions are combined with AND, not OR. `indexOf` returns -1 when the character is missing, and 0 is a valid position (the first character). Off-by-one errors in length checks (`>` vs `>=`) are a classic bug.

1. The first check must be `isUpperCase`
2. `indexOf(c) > 0` misses `!` → `>= 0`
3. `length() > 8` → `>= 8`
4. All four classes are required: `&&`, not `||`

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String p = new Scanner(System.in).next();
        String specials = "!@#$%^&*";
        boolean up = false, low = false, dig = false, sp = false;
        for (char c : p.toCharArray()) {
            if (Character.toUpperCase(c)) up = true;
            else if (Character.toLowerCase(c)) low = true;
            else if (Character.isDigit(c)) dig = true;
            else if (specials.indexOf(c) >= 0) sp = true;
        }
        boolean strong = p.length() >= 8 && (up && low && dig && sp);
        System.out.println(strong ? "STRONG" : "WEAK");
    }
}
```

H1 (C++)

Concept explained: Dijkstra's algorithm repeatedly settles the closest unsettled node, using a MIN-priority queue. C++ `priority_queue` is a max-heap by default. Use `greater<>`, or push negated distances. Stale heap entries (`d > dist[u]`) must be skipped, otherwise it runs too slowly. An undirected edge needs both directions. Use 64-bit integers for distances. Dijkstra is the basis of routing protocols like OSPF.

1. The graph is undirected, so add the reverse edge
2. `priority_queue` is a max-heap, so use `greater<>`
3. Skip stale entries (`if (d > dist[u]) continue;`), otherwise TLE
4. `int / INT_MAX` overflows (`d + w`, and paths up to $2 \cdot 10^{14}$), so use `long long`

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<pair<int, long long>>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v; long long w; cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    const long long INF = LLONG_MAX;
    vector<long long> dist(n + 1, INF);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
    dist[1] = 0; pq.push({0, 1});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    for (int i = 1; i <= n; i++) cout << (dist[i] == INF ? -1 : dist[i]) << " \n"[i ==
n];
}

```

H2 (Python)

Concept explained: Minimize the MSE $J = (1/n)\sum(w x + b - y)^2$. Its gradients are $\partial J / \partial w \propto (1/n)\sum \text{err} \cdot x$ and $\partial J / \partial b \propto (1/n)\sum \text{err}$. The update moves AGAINST the gradient: $w \leftarrow w - \text{lr} \cdot \partial w$. The gradient sums must be reset at the start of every iteration, otherwise they keep accumulating.

1. Gradients must be reset **inside** each iteration
2. `w += ...` $\rightarrow$ `w -= ...` (the sign is wrong, so it diverges)
3. The `b` gradient is missing `/ n`

```

n, T, lr = input().split()
n, T, lr = int(n), int(T), float(lr)
xs, ys = [], []
for _ in range(n):
    x, y = map(float, input().split())
    xs.append(x); ys.append(y)
w = b = 0.0
for _ in range(T):
    dw = db = 0.0
    for i in range(n):
        err = (w * xs[i] + b) - ys[i]
        dw += err * xs[i]
        db += err
    w -= lr * dw / n
    b -= lr * db / n
print(f"{w:.4f} {b:.4f}")

```

H3 (Java)

Concept explained: An LRU cache evicts the least recently used key, and CPUs, databases and OSes use the same policy. Java's `LinkedHashMap(cap, 1f, accessOrder=true)` keeps entries

in usage order, and `removeEldestEntry` evicts when `size > capacity`. Unboxing a null Integer throws a `NullPointerException`, so use `getOrDefault`.

1. `accessOrder` must be `true`, otherwise it's FIFO, not LRU
2. `size() >= cap` evicts too early → `size() > cap`
3. `get` on a missing key returns null, and unboxing it throws an NPE, so use `getOrDefault(key, -1)`. With `accessOrder=true`, `getOrDefault` still counts as an access.

```
import java.util.*;
public class Main {
    static class LRU extends LinkedHashMap<Integer, Integer> {
        int cap;
        LRU(int cap) { super(16, 0.75f, true); this.cap = cap; }
        protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) { return
size() > cap; }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int c = sc.nextInt(), q = sc.nextInt();
        LRU cache = new LRU(c);
        StringBuilder out = new StringBuilder();
        while (q-- > 0) {
            String op = sc.next();
            if (op.equals("GET")) {
                int v = cache.getOrDefault(sc.nextInt(), -1);
                out.append(v).append('\n');
            } else cache.put(sc.nextInt(), sc.nextInt());
        }
        System.out.print(out);
    }
}
```

H4 (C++)

Concept explained: $y[n] = \sum_k h[k] \cdot x[n-k]$. Full linear convolution of lengths N and M has length $N+M-1$. The valid range is $0 \leq n-k < N$. Every digital filter, and every CNN layer, is a convolution. Use 64-bit accumulators to avoid overflow.

1. Output length is `N + M - 1`
2. `n - k > 0` → `>= 0` (misses `x[0]`)
3. Products reach 10^{12} and sums $2 \cdot 10^{15}$, so use `long long`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M; cin >> N >> M;
    vector<long long> x(N), h(M);
    for (auto &v : x) cin >> v;
    for (auto &v : h) cin >> v;
    int L = N + M - 1;
    vector<long long> y(L, 0);
    for (int n = 0; n < L; n++)
        for (int k = 0; k < M; k++)
            if (n - k >= 0 && n - k < N) y[n] += h[k] * x[n - k];
    for (int i = 0; i < L; i++) cout << y[i] << " \n"[i == L - 1];
}
```

H5 (Python)

Concept explained: RSA: $n = pq$, $\phi = (p-1)(q-1)$, $d = e^{-1} \bmod \phi$, $m = c^d \bmod n$. Extended Euclid finds x with $ex \equiv 1 \pmod{\phi}$, and x may be negative, so return $x \bmod \phi$. Square-and-multiply exponentiation needs $b = b*b \% m$ and $e \gg= 1$, which takes $O(\log e)$ steps.

1. `inv` can return a negative x , so return $x \% m$
2. $b = b * b \rightarrow b = b * b \% m$ (numbers explode)
3. $e \gg 1$ does nothing (infinite loop) $\rightarrow e \gg= 1$
4. $\phi(n) = (p - 1) * (q - 1)$

```
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = egcd(b, a % b)
    return g, y, x - (a // b) * y

def inv(a, m):
    g, x, _ = egcd(a, m)
    return x % m

def power(b, e, m):
    r = 1
    b %= m
    while e > 0:
        if e & 1:
            r = r * b % m
        b = b * b % m
        e >>= 1
    return r

p, q, e, c = map(int, input().split())
n = p * q
phi = (p - 1) * (q - 1)
d = inv(e, phi)
print(power(c, d, n))
```

H6 (Java)

Concept explained: Kahn's algorithm repeatedly outputs a node with in-degree 0 and lowers the in-degree of its successors. For edge $u \rightarrow v$, v 's in-degree increases. A min-heap gives the lexicographically smallest order. If fewer than n nodes are output, the graph contains a cycle.

1. The in-degree must be incremented for v , not u
2. A stack doesn't give the lexicographically smallest order, so use `PriorityQueue` (offer/poll)
3. Missing cycle detection: if `order.size() < n`, print `CYCLE`

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), m = sc.nextInt();
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i <= n; i++) g.add(new ArrayList<>());
        int[] indeg = new int[n + 1];
        for (int i = 0; i < m; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            g.get(u).add(v);
            indeg[v]++;
        }
        PriorityQueue<Integer> q = new PriorityQueue<>();
        for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.offer(i);
        List<Integer> order = new ArrayList<>();
        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
        }
        if (order.size() < n) { System.out.println("CYCLE"); return; }
        StringBuilder sb = new StringBuilder();
        for (int x : order) sb.append(x).append(' ');
        System.out.println(sb.toString().trim());
    }
}

```

H7 (C++)

Concept explained: Each full adder computes sum bit = $(a+b+cin) \bmod 2$ and carry = $(a+b+cin)/2$, and the carry ripples to the next position. The loop must continue while either input has bits left or a carry remains. Characters must be converted to digits with `'0'`. The result is built LSB-first, so reverse it.

1. The loop must continue while **either** string has digits or there is a carry: `i >= 0 || j >= 0 || carry`
2. Characters must be converted: `a[i--] - '0'`, `b[j--] - '0'`
3. The sum bit is `sum % 2` and the carry is `sum / 2` (they're swapped)
4. Bits are produced LSB-first, so reverse the result

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string a, b; cin >> a >> b;
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    string res;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';
        res.push_back('0' + sum % 2);
        carry = sum / 2;
    }
    reverse(res.begin(), res.end());
    cout << res << endl;
}

```

H8 (Python)

Concept explained: K-means alternates two steps: (1) assign each point to its NEAREST centroid, (2) recompute each centroid as the mean of its cluster. These steps must stay separate, so centroids aren't updated during assignment. `[[[]]*k` creates aliased lists. An empty cluster would divide by zero, so it keeps its old centroid.

1. `[[[]] * k` creates aliased lists
2. `>` picks the **farthest** centroid → `<` (strict keeps the lower index on a tie)
3. Centroids must not be updated inside the assignment loop (remove that line)
4. An empty cluster divides by zero, so it must keep its old centroid

```

n, k, T = map(int, input().split())
pts = list(map(int, input().split()))
cent = [float(v) for v in pts[:k]]
for _ in range(T):
    clusters = [[] for _ in range(k)]
    for p in pts:
        best = 0
        for c in range(1, k):
            if abs(p - cent[c]) < abs(p - cent[best]):
                best = c
        clusters[best].append(p)
    for c in range(k):
        if clusters[c]:
            cent[c] = sum(clusters[c]) / len(clusters[c])
print(" ".join(f"{v:.2f}" for v in sorted(cent)))

```

H9 (Java)

Concept explained: Keep the deque's indices in decreasing order of value. Drop the front index when it falls out of the window (`index <= i - k`). Pop from the back every value `<= a[i]`, because those can never be a maximum again. The first full window ends at `i = k - 1`. The whole algorithm is $O(n)$.

1. The expiry check must be `dq.peekFirst() <= i - k`
2. Pop smaller-or-equal elements from the back: `a[dq.peekLast()] <= a[i]`

3. The first full window ends at $i = k - 1 \rightarrow i \geq k - 1$

```
import java.util.*;
import java.io.*;
public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken()), k = Integer.parseInt(st.nextToken());
        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());
        Deque<Integer> dq = new ArrayDeque<>();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
            while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();
            dq.addLast(i);
            if (i >= k - 1) sb.append(a[dq.peekFirst()]).append(' ');
        }
        System.out.println(sb.toString().trim());
    }
}
```

H10 (C++)

Concept explained: $dp[i][j]$ is the edit distance between the first i characters of s and the first j characters of t . Base cases: $dp[i][0]=i$ and $dp[0][j]=j$. Recurrence: $dp[i][j] = \min(\text{delete } dp[i-1][j]+1, \text{insert } dp[i][j-1]+1, \text{replace or match } dp[i-1][j-1]+(s[i-1] \neq t[j-1]))$. It is used in spell-checkers and DNA sequence alignment.

1. The DP table must be $(n+1) \times (m+1)$
2. Base cases are missing: $dp[i][0] = i$, $dp[0][j] = j$
3. Off-by-one indexing: compare $s[i-1]$ with $t[j-1]$
4. The cost is inverted: equal characters cost **0**
5. The insertion option $dp[i][j-1] + 1$ is missing from the `min`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s, t; cin >> s >> t;
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            int cost = (s[i - 1] == t[j - 1]) ? 0 : 1;
            dp[i][j] = min({dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] +
cost});
        }
    cout << dp[n][m] << endl;
}
```